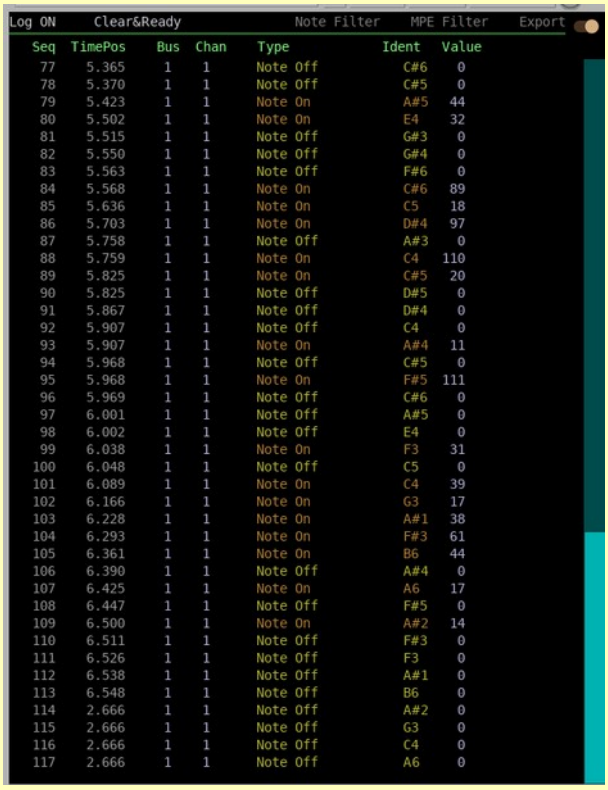
MidiLog

tjingboem productions
september 2026

Overview

MidiLog is a MIDI-monitoring and analysis tool for REAPER -- an oscilloscope for MIDI data. It logs every message passing through an FX chain (Note On/Off, CC, Pitch Wheel, Channel Pressure, Poly Aftertouch, SysEx) with sample-accurate timing, shows it scrolling live on screen as it happens, and can export the entire captured history to a text file and an interactive HTML page for closer analysis afterward.

It isn't tied to any particular project, plugin, or workflow -- drop it anywhere in an FX chain you want visibility into. Debugging an MPE controller, checking exactly what a generative script sent, or just wanting to see MIDI traffic as it happens are all the same use to MidiLog: it doesn't care why you're watching, it just watches.



Seq	TimePos	Bus	Chan	Type	Ident	Value
77	5.365	1	1	Note Off	C#6	0
78	5.370	1	1	Note Off	C#5	0
79	5.423	1	1	Note On	A#5	44
80	5.502	1	1	Note On	E4	32
81	5.515	1	1	Note Off	G#3	0
82	5.550	1	1	Note Off	G#4	0
83	5.563	1	1	Note Off	F#6	0
84	5.568	1	1	Note On	C#6	89
85	5.636	1	1	Note On	C5	18
86	5.703	1	1	Note On	D#4	97
87	5.758	1	1	Note Off	A#3	0
88	5.759	1	1	Note On	C4	110
89	5.825	1	1	Note On	C#5	20
90	5.825	1	1	Note Off	D#5	0
91	5.867	1	1	Note Off	D#4	0
92	5.907	1	1	Note Off	C4	0
93	5.907	1	1	Note On	A#4	11
94	5.968	1	1	Note Off	C#5	0
95	5.968	1	1	Note On	F#5	111
96	5.969	1	1	Note Off	C#6	0
97	6.001	1	1	Note Off	A#5	0
98	6.002	1	1	Note Off	E4	0
99	6.038	1	1	Note On	F3	31
100	6.048	1	1	Note Off	C5	0
101	6.089	1	1	Note On	C4	39
102	6.166	1	1	Note On	G3	17
103	6.228	1	1	Note On	A#1	38
104	6.293	1	1	Note On	F#3	61
105	6.361	1	1	Note On	B6	44
106	6.390	1	1	Note Off	A#4	0
107	6.425	1	1	Note On	A6	17
108	6.447	1	1	Note Off	F#5	0
109	6.500	1	1	Note On	A#2	14
110	6.511	1	1	Note Off	F#3	0
111	6.526	1	1	Note Off	F3	0
112	6.538	1	1	Note Off	A#1	0
113	6.548	1	1	Note Off	B6	0
114	2.666	1	1	Note Off	A#2	0
115	2.666	1	1	Note Off	G3	0
116	2.666	1	1	Note Off	C4	0
117	2.666	1	1	Note Off	A6	0

Two Files, Not One

MidiLog is two separate files working together, not a single plugin. MidiLog.jsfx is the effect itself -- the thing you actually load onto a track, showing the live scrolling log on screen. Log_Export.lua is a small companion script that has to run alongside it, in the background, to actually save an export to disk.

This split exists because of a real limitation, not a design choice: the JSFX format MidiLog.jsfx is built in cannot write files to disk, and cannot call REAPER's own scripting API directly. So the moment you click Export, the JSFX side hands its captured data off through a shared memory channel, and the Lua side -- which can do both those things -- is what actually writes the file.

Both pieces need to be running for Export to work. The on-screen log itself works with just the JSFX loaded -- you only need Log_Export.lua running at the moment you want to save something. "Getting Started" below walks through setting up both.

Getting Started

Loading the Effect

Add MidiLog.jsfx to any track's FX chain the way you'd add any REAPER effect -- the FX menu on the track, then search for MidiLog. Where you place it in the chain matters: put it first to see the raw MIDI arriving at the track, or after another MIDI-processing plugin to see exactly what that plugin is actually sending onward. MidiLog passes everything through unchanged; it only watches.

Starting the Export Watcher

Log_Export.lua has to be launched separately, once, via REAPER's Actions list (Actions -> Show action list... -> New action... -> Load ReaScript..., then find and run Log_Export.lua). Nothing visible happens when it starts -- no window opens. It's a background watcher: from that moment on, it sits quietly checking for an Export click, and writes a file the instant one happens.

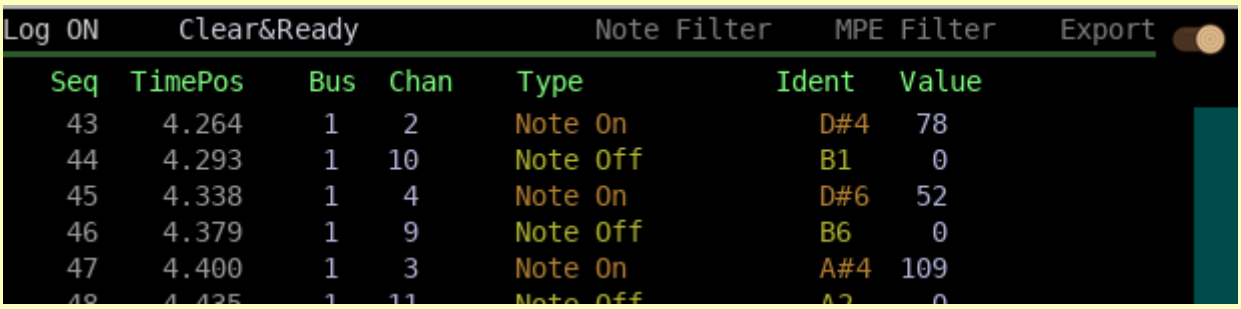
It has to stay running. Closing it, or restarting REAPER, stops it -- you'll need to launch it again before your next Export will actually produce a file. If you want it running every time REAPER opens without doing this by hand, REAPER's Actions list lets you set any loaded action as a Global Startup Action -- worth doing once you know you'll be using MidiLog regularly.

Confirming It's Working

Click "Log OFF" so it switches to "Log" (brighter, active). Play a few notes on the track. Click "Export". A message box should appear showing the exact file path it just wrote, for example .../MidiLogExports/midilog_20260909_153000.txt. If nothing happens when you click Export, the almost-certain cause is that Log_Export.lua isn't running -- go back and launch it.

The On-Screen Window

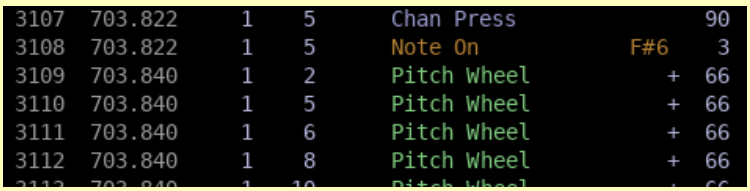
Along the top of the window, five buttons; below them, the scrolling log itself.



Log ON / Log OFF -- turns MIDI capture on and off. Brighter means active. The label itself switches text, not just color, so the state is readable at a glance. While off, nothing new is added to the history -- everything already captured stays visible and exportable.

Clear&Ready -- wipes the history and resets the Elapsed-time origin (see "Timing" below) in one click. Named for what it actually does: not just clearing the screen, but marking that a fresh measurement is about to begin.

Note Filter -- hides Note On/Off rows from the on-screen view only. Export always ignores this and captures everything -- see "Exporting" below.



MPE Filter -- hides Pitch Wheel, Channel Pressure, and CC74 rows (the three MPE expression dimensions) from the on-screen view only, with the same Export exception. A repeated MPE value that's an exact duplicate of the previous value on its own channel is also hidden automatically

whenever MPE data is visible at all -- a receiver holding a value doesn't need announcing every block.

Export -- dumps the entire captured history, ignoring both filters above, to a text file and an HTML page. See "Exporting" below.



You'll also see a small log icon in the top-right corner of this row, next to Export -- the same image used to mark Log_Export.lua as MidiLog's companion script (see "Getting Started" above). A copy of it lives in the MidiLog folder itself (MidiLog.png, and MidiLog_toolbar.png for a REAPER toolbar button) if you want to use it yourself -- for example, assigning it to Log_Export.lua's own toolbar button, so the two files stay visually tied together wherever you launch the script from.

The Log Table

Below the buttons: a scrollable, auto-following log with columns for Seq, TimePos, Bus, Chan, Type, Ident, and Value, a right-hand scrollbar you can drag, and click-to-jump on the scrollbar track. Auto-follow (staying stuck to the latest entry) is filter-aware -- it counts only currently visible rows when deciding what's in view, so filtering out a dominant message type doesn't leave the view starved down to almost nothing.

Timing: TimePos vs. Elapsed

Two independent time values are captured per event, both shown in the export.

TimePos -- REAPER's own play-cursor position. It only moves while the transport is actually rolling -- useless as a time axis if you're just holding keys live with playback stopped, since it sits frozen the whole time.

Elapsed -- real wall-clock time, independent of transport state, measured from your last Clear&Ready click. This is the axis every diagram and the Duration line are built on, since it always means something regardless of whether you were pressing Play.

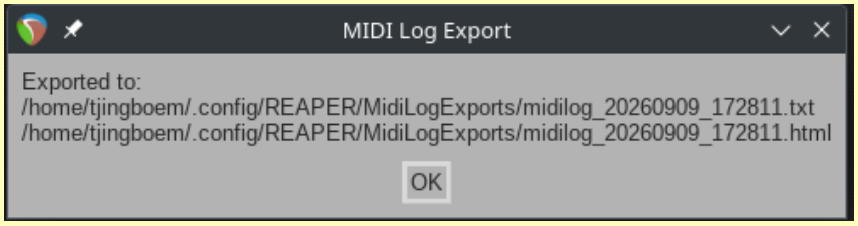
TimePos	Elapsed
12.009	0.000
12.009	0.000
12.009	0.000
12.009	0.000
12.000	0.022
12.000	0.022
12.001	0.022
12.001	0.022
12.001	0.022
12.000	0.043

In an export, Elapsed is further zero-based to the very first captured event -- there's usually a gap of a second or more between clicking Clear&Ready and playing your first note, and subtracting that gap out means "0.000" in the file always means "the first note," not "whenever you happened to click Clear&Ready."

Duration, the summary line at the top of every export, is the span from the first captured event to the last one -- not "whatever time it happens to be when you click Export." So Duration stays correct whether Log is still on, was switched off partway through, or Export was clicked long after you actually stopped playing.

Exporting

Clicking Export always dumps the full captured history, ignoring the Note and MPE filters -- those are a viewing convenience for the on-screen log, not a limit on what actually gets saved.



Each Export writes two files into <REAPER resource path>/MidiLogExports/: a midilog_YYYYMMDD_HHMMSS.txt (the raw log plus the ASCII diagrams) and a midilog_YYYYMMDD_HHMMSS.html (the same capture as an interactive drawing). A message box appears with the exact path, naming both files -- this is the actual confirmation that something happened, not just a console line you might miss.

Reading an Export

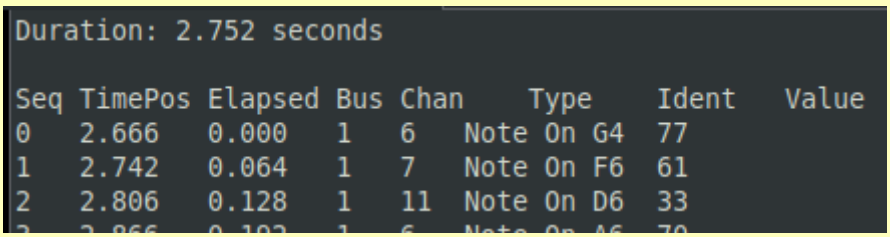
An export has two parts: the raw log (a tab-separated table) and, three blank lines below a "--- Diagrams ---" header, the ASCII diagrams. Every export carries its own short legend directly under that header, covering only the glyphs that actually appear in that particular file -- so for most reading you won't need this chapter at all. What follows is the complete reference, including cases a given export happens not to contain.

The Raw Log

One line per MIDI message, tab-separated, under a Duration line giving the length of the capture in seconds.

Column	Meaning
Seq	Message number, from 0, in arrival order.
TimePos	Project time in seconds.
Elapsed	Seconds since the first message after Clear -- the same clock every diagram is drawn against.
Bus	MIDI bus.
Chan	MIDI channel, 1-16.
Type	Note On, Note Off, CC, Pitch Wheel, Chan Press, Poly AT, Program, SysEx.
Ident	What the message is about: a note name (A#1), or a CC number and name (74 Sound Brightness). Blank for messages with no identity of their own, such as Pitch Wheel and Channel Pressure.
Value	Velocity, CC value, pressure, or bend.

A Note Off is logged as such, but note that many controllers send note-off as Note On with Value 0 -- both appear, and the diagrams treat them identically.

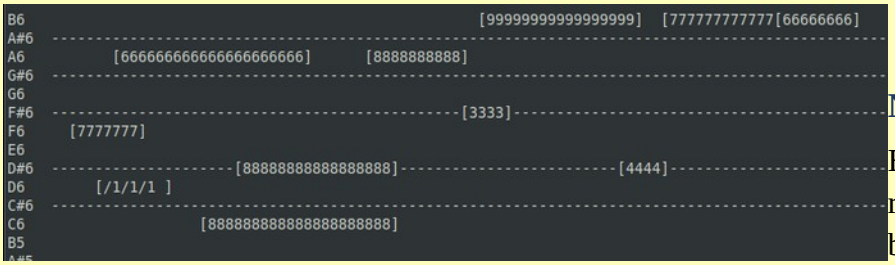


Keyboard Diagram (Notes)

One row per semitone, highest pitch at the top, including semitones that were never played -- a real keyboard has no gaps, so the chromatic spacing stays true and a wrong note is visible by position alone. All channels share one diagram rather than getting one each.

Symbol	Meaning
[	A note's onset.
]	A note's release.
1-9	Bar fill for channels 1-9; the digit is the channel.
/N	Bar fill for channels 10-16, as a two-character unit: /0 = channel 10, /3 = channel 13, /6 = channel 16.
-	Background of a black-key row (C#, D#, F#, G#, A#).
(space)	Background of a white-key row.

So [888] is one note on channel 8, and [8][8] is two separate notes back to back -- the brackets are what tell those apart. [/3/3] is one note on channel 13. Channels 10-16 use /N rather than two digits so a repeated single digit can never be misread: 88 is unambiguously two columns of channel 8, never channel 88.



Narrow Bars

Brackets are reserved out of the bar's own width, so a short note has to give something up:

Bar width	Renders as	Kept
1 column	8	Channel only, no brackets.
2 columns	[8	Onset bracket; release dropped.
3+ columns	[8], [88], ...	Both brackets, channel fill between.

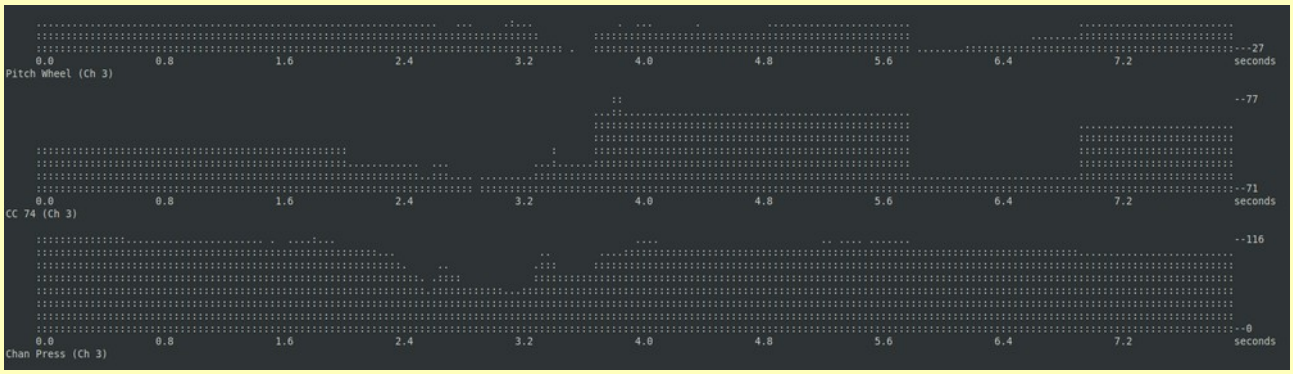
For a channel 10-16 bar whose inner space is too narrow for one complete /N unit, that space is left as background rather than showing half a symbol. A note still sounding when Export was pressed has no release, and its bar runs to the right edge with no].

Value Diagrams (CC, Pitch Wheel, Channel Pressure, Poly Aftertouch)

One diagram per (type, identity, channel) combination -- CC74 on channel 2 and CC74 on channel 3 are two separate diagrams, never merged, because MPE spreads one gesture across many channels and merging would hide which channel actually moved. Eight rows tall, drawn as an area chart -- everything below the current value is filled.

Symbol	Meaning
:	Row fully reached by the value.
.	Row half reached -- doubles the vertical resolution from 8 steps to 16.
(space)	Above the current value.
--123 (top row)	The highest value in this diagram.
--123 (bottom row)	The lowest value in this diagram.
label	Under the diagram, e.g. "CC 74 (Ch 2)".

The vertical scale auto-fits each diagram's own range rather than spanning a fixed 0-127, so a small movement is still visible as a shape. That's why the hi/lo numbers are printed -- without them there'd be no way to know what the top and bottom of a given chart actually mean. A flat line with the same number top and bottom is a value that never changed. Values are sample-and-hold: the line holds its last value until the next message, which is how MIDI actually behaves.



The Time Ruler

The row of numbers under each diagram marks seconds since the first message, labelled at regular intervals and followed by the word "seconds". Every diagram in one export shares the same width and the same left margin, so a column is the same instant in every diagram -- you can read straight down from a note in the keyboard diagram into the CC diagrams below it and see what the expression was doing at that exact moment.

Panels

The grid width is chosen per export from the capture's own duration, targeting about 25 ms per column and clamped to between 100 and 400 columns -- a short capture is drawn finely, a long one coarsely.

A wide grid would produce lines too long for an editor window, and every pitch row is the same length, so they'd all wrap at the same point and interleave -- destroying the vertical alignment the diagram exists for. Instead the timeline is cut into panels of at most 200 columns, stacked down the page like systems on a musical score. Each panel is a complete keyboard followed by the value lanes for that same span of time, and the series continues from where the previous panel stopped -- times on the ruler are absolute, so a later panel picks up reading where the one before it left off, rather than restarting at zero.

Every pitch row is drawn in every panel, including pitches silent in that slice, so the keyboard keeps its shape and panels can be compared by eye. A value lane, on the other hand, appears only in panels where that stream actually moved -- a channel doing nothing in a given panel contributes no lane there. Value lanes are scaled to the stream's range across the whole capture, not per panel, so a given height means the same thing everywhere -- the cost is that a gesture small relative to the whole take can look flat in its own panel.

The HTML Page

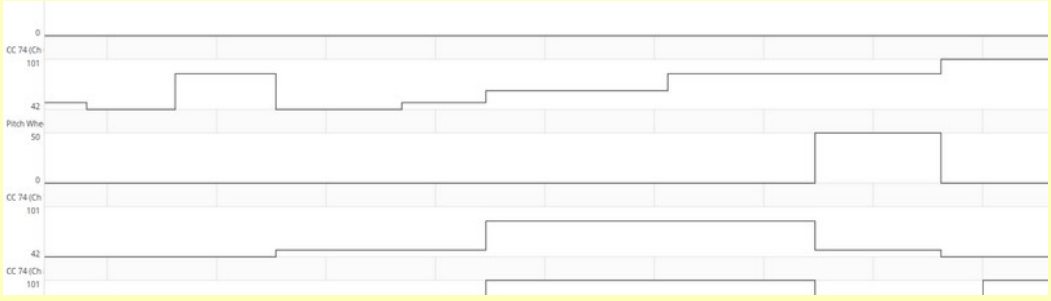
Every export also writes an .html file beside the .txt, holding the same capture as a drawing -- open it in any browser.



Here, every legibility compromise above -- bracket widths, /N units, panels, collisions -- comes from the fixed grid of a text file. A drawing has none of those limits: notes become rectangles at their true start and end, and the timeline is one continuous scroll.

Element	Meaning
Bar colour	The note's channel -- one of sixteen.
Dark left tick	The note's onset, the equivalent of [.
Gap between bars	A note boundary, not necessarily a rest.
Shaded rows	Black keys, as in the ASCII diagram.
Line under roll	A value stream, sharing the same time axis.
Footer	Streams that never moved, and their value.

The onset tick and the gap both exist for the same reason the brackets do in the text version: consecutive notes on one pitch and channel are the same colour, and without them five notes in a row would read as one long note. Hover any note for its pitch, channel, velocity, and duration in milliseconds -- velocity appears nowhere in the ASCII diagrams, which have no room for it. Channel checkboxes in the header hide or isolate channels, which is the fastest way to check what one part of a multitimbral fold actually received.



The page is written in addition to the text, never instead of it -- the .txt stays greppable, diffable, and pasteable, which the page is not. If building the page ever fails, the text export is unaffected and the confirmation dialog simply won't mention it.

Support MidiLog

MidiLog is donationware: free to use, with no restrictions, for as long as you find it useful. If it's earned a place in your workflow, a donation is always appreciated but never required.